

Web services with Python (L04)

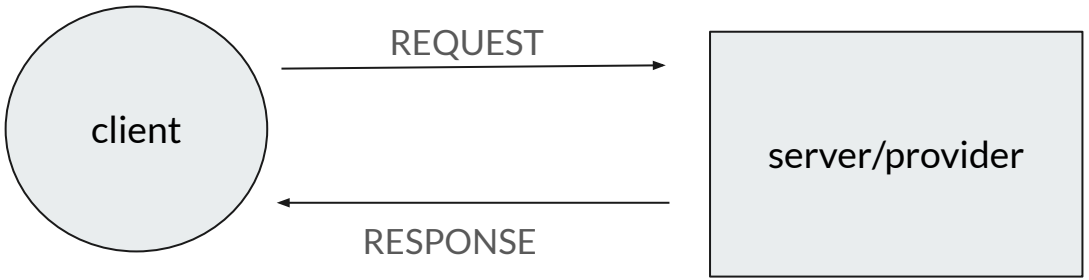
Web services

examples:
You use a weather app → it calls a **weather web service** to get today's forecast.

You book a hotel → the app calls a **booking web service** to check availability.

A web service allows two application to **interact** over the internet usually using the HTTP protocol.

When we talk about web services (in this course) we **ALWAYS** implicitly refer to a specific communication paradigm → **Request/Response (synchronous)**



- web servers alway up and running
- web servers expose services (resources)
- web client consumes services exposed by the web server
- web client start the communication

API - Application Programming Interface

An API defines how one piece of software can interact with another, what can be asked, how to ask for it and what will be the expected return.

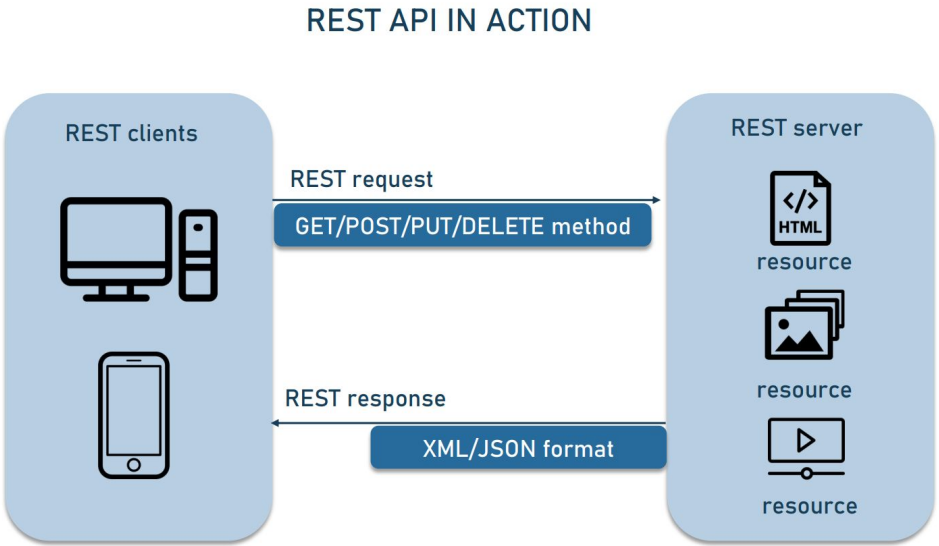
API —> defines functions or endpoints that you can call

Action	HTTP Method	Endpoint	Description
List houses	`GET`	`/houses`	All houses with their rooms
One house	`GET`	`/houses/{h}`	Rooms of one house
Register a device	`POST`	`/devices`	Adds or replaces a device, returns its config
Change thresholds	`PUT`	`/houses/{h}/{r}/thresholds`	New t_min, t_max, h_max

REST - representational state transfer

REST is an architecture style that defines some constraints for designing web services.

REST says: “Let’s use the web as it’s meant to be used — resources identified by URLs, and standard HTTP methods to interact with them.”



HTTP methods and How to use them RESTful

Method	Action	CRUD	Typical Use
GET	Retrieve a resource	Read	Fetch data
POST	Create a new resource	Create	Submit new data
PUT	Replace or update a resource	Update	Modify existing data
DELETE	Remove a resource	Delete	Delete data

POST and PUT use a body, GET and DELETE no!

The client side in Python

The client performs requests! to perform requests in Python we use the **requests** library.

- pip install requests

```
r = requests.get(url, params={"a": 1}, timeout=10)
r.raise_for_status()      # error for 4xx/5xx
data = r.json()
r = requests.post(url, json={"x": 1}, timeout=10)
```

Always set a **timeout**. **json=** sets the body and the Content-Type header.

Using a web-service

full status code list

This github repository contains a list of public webservices with an API documentation that could be used! have fun!

- choose one
- read API documentation (to understand how to structure a request and what kind of request are supported)
- create a small python script in which you perform the request and see the response
- RESPONSE STATUS: when you perform a request is not granted a success, thus the first thing to check is the code of response status, there exist standards for that.

Status codes you will meet:
200 OK
201 Created
400 Bad request (client error)
404 Not found
500 Server error

Query string = optional "what to look for". JSON body = structured data to create or change.

Exercise 1 - CityBikes client

Goal: practise a REST client with requests: call a public API, read the JSON, handle errors.

Task: get the bike-sharing networks of a city from https://api.citybik.es/v2/networks (city from the command line, default Milano).

For each network, GET its stations (href field) and print name, free bikes and empty slots.

Do not crash on timeouts or HTTP errors; print a message if no network matches.

File: L04_rest_basics/solution/ex1_citybikes_client.py (no network for Torino: try Milano, Barcelona, Paris)

Designing & implementing a web-service

We will use the cherrypy library to define and implement web-services.

- first install cherrypy
- Run hello_world.py (2026_2027/L04_rest_basics/solution/): GET / returns Hello World; GET /?name=Anna returns Hello Anna. Test with the browser and curl.

RESTful: main REST principles ref: IBM REST API or medium

- **Uniform interface**
- **Client-Server decoupling**
- **Statelesseness**
- **Cacheability**
- **Layered architecture**
- **Code on demand**

CherryPy in 6 lines

- A class with exposed = True is a resource; MethodDispatcher maps the HTTP method to GET/POST/PUT/DELETE.
- GET(self, *uri, **params): uri = path segments, params = query string (always strings).
- @cherrypy.tools.json_out() / json_in() for JSON; errors: raise cherrypy.HTTPError(400, "message").

```
class Hello:
    exposed = True

    @cherrypy.tools.json_out()
    def GET(self, *uri, **params):
        return {"hello": params.get("name", "World")}
```

Exercise 2 - calculator service

Goal: write your first CherryPy web service: GET and POST, JSON answers, explicit errors.

- Task: a calculator service.
- GET /add?a=1&b=2 (also sub, mul, div) → {"operation": "add", "a": 1.0, "b": 2.0, "result": 3.0}
- POST / with {"operation": "add", "operands": [1, 2, 3]} → same fields, result over all operands (6.0)
- Wrong input (missing or non-numeric value, unknown operation, division by zero) → 400 with a clear message; unknown path → 404
- Then write a client with requests that tests GET, POST and the errors.
- Files: L04_rest_basics/solution/ex2_calculator.py, ex2_client.py

Immediate rejection and good practices

Immediate rejection

- Not microservices (a monolith)
- Not object-oriented

Good practices

- JSON data format
- Add/remove devices, users, contexts at runtime, no code changes
- JSON configuration files, no hard-coded values
- Communication only over REST and/or MQTT

The course smart-home project follows exactly these rules: use it as a model.